

module3_lab

September 16, 2026

```
[1]: import pandas as pd

df = pd.read_csv("messy_sales.csv")
print(df.shape)
df.head(10)
```

(300, 6)

```
[1]:   order_id      date  product  price  qty  zip
0      1254  04/06/2026      mug   11.36    4  60614
1      1057  2026-05-30  charger   14.79    3  30303
2      1150  05/06/2026      mug    5.50    1  10001
3      1066  05/30/2026  notebook    NaN    1  98101
4      1114  04/06/2026  webcam   45.59    4  90405
5      1280  04/22/2026  pen set    NaN    4  30303
6      1204  04/18/2026  charger    8.41    2   2134
7      1249  2026-05-15  keyboard   75.04    4  90405
8      1037  2026-05-19      mug    9.31    2  60614
9      1177  2026-05-13      mug   10.11    3  90210
```

```
[2]: df.dtypes
```

```
[2]: order_id      int64
date           str
product        str
price         float64
qty           int64
zip           int64
dtype: object
```

0.1 Observations

1. The date column mixes two different formats — some dates are written as day/month/year and others as year/month/day, with the order reversed. This means the system can't reliably tell which number is the day, month, or year.
2. Zip codes lost their leading zero, which changes the value and shows an incorrect number.
3. Some rows have a missing (empty) price value, which would break calculations like total revenue.

```
[3]: df.isna().sum()
```

```
[3]: order_id      0
     date          0
     product       0
     price        12
     qty           0
     zip           0
     dtype: int64
```

```
[4]: fill_value = df["price"].median()
     print(fill_value)

     df["price"] = df["price"].fillna(fill_value)
```

```
37.53
```

```
[5]: print(df.duplicated().sum())
     df = df.drop_duplicates()
     print(df.shape)
```

```
8
(292, 6)
```

```
[6]: df["zip"] = df["zip"].astype(str).str.zfill(5)
     df["zip"].head(10)
```

```
[6]: 0    60614
     1    30303
     2    10001
     3    98101
     4    90405
     5    30303
     6    02134
     7    90405
     8    60614
     9    90210
     Name: zip, dtype: str
```

```
[7]: df["date"] = pd.to_datetime(df["date"], format="mixed")
     df.dtypes
```

```
[7]: order_id      int64
     date          datetime64[us]
     product       str
     price         float64
     qty           int64
     zip           str
     dtype: object
```

```
[8]: print(df.shape)
      print(df["zip"].str.len().unique())
      print(df.dtypes)
```

```
(292, 6)
[5]
order_id      int64
date          datetime64[us]
product       str
price         float64
qty           int64
zip           str
dtype: object
```

```
[9]: df[df["qty"] < 0]
```

```
[9]:   order_id   date  product  price  qty  zip
202     1140 2026-04-18  webcam  38.69  -5  98101
262     1233 2026-05-09  desk lamp  22.64  -4  10001
297     1025 2026-04-27  keyboard  47.30  -2  02116
```

```
[10]: df["qty"] = df["qty"].abs()
```

```
[11]: df[df["qty"] < 0]
```

```
[11]: Empty DataFrame
      Columns: [order_id, date, product, price, qty, zip]
      Index: []
```

I noticed that the three rows with negative qty appear across three different products. I assumed it was more likely that these were three separate typing errors, rather than three different customers happening to return three different products. So I used `abs()` to convert the values to positive, since if it's a typo, the true quantity would be positive.

```
[12]: df.to_csv("sales_clean.csv", index=False)
```

```
[13]: import os
      "sales_clean.csv" in os.listdir()
```

```
[13]: True
```

0.2 Cleaning Log

- Loaded 300 rows, 6 columns.
- Missing prices: 12 rows were missing a price value. Filled using the median price, 37.53 (median was chosen instead of the mean because price is skewed by a few high-value products).
- Duplicates: 8 duplicate rows were found and removed, leaving 292 rows.
- Zip codes: converted from int64 to text and restored to 5 characters with leading zeros (e.g. 2134 → 02134).

- Dates: standardized to real datetime values. The column previously mixed two formats (day/month/year and year/month/day), which prevented sorting or filtering by date.
- Negative quantities: 3 rows had negative qty values, each for a different product. I assumed it was more likely that these were three separate typing errors, rather than three different customers happening to return three different products. So I used `abs()` to convert the values to positive, since if it's a typo, the true quantity would be positive.
- Final shape: (292, 6).

If we leave the duplicate rows in, the total revenue calculation will come out inflated and inaccurate, which could lead a manager to set a budget that's too high and doesn't match reality.

[]: